



UNIWERSYTET VIZJA

KIERUNEK: INFORMATYKA

Roland Piątkowski

studia niestacjonarne

nr albumu 46653

**Projekt i implementacja systemu wspomagania zarządzania
korespondencją i terminami w postępowaniach
administracyjnych**

Dokumentacja do projektu dyplomowego

przygotowana pod kierunkiem

dra Marcina Kacprowicza

Warszawa, 2026

1. Podstawowe informacje

Nazwa projektu

Projekt i implementacja systemu wspomagania zarządzania korespondencją i terminami w postępowaniach administracyjnych.

Cel projektu

W postępowaniach prowadzonych z instytucjami obowiązują terminy o skutkach nieodwracalnych. Ich przekroczenie powoduje utratę prawa do wniesienia środka odwoławczego, pozostawienie sprawy bez rozpoznania albo powstanie zobowiązania finansowego. Liczy się je od daty doręczenia pisma, nie od daty jego nadania, a długość i sposób liczenia różnią się w zależności od czynności.

Mimo tych konsekwencji sprawy prowadzi się zwykle w folderach i arkuszach kalkulacyjnych, a terminów pilnuje pamięcią albo przypomnieniami w kalendarzu. Sposób ten nie skaluje się przy kilku równoległych sprawach i nie zabezpiecza przed pomyłką w samym liczeniu — a pomyłka o jeden dzień bywa równie dotkliwa jak całkowite zaniedbanie sprawy.

Celem projektu jest aplikacja webowa, która gromadzi sprawy, korespondencję i dokumenty w jednym miejscu, wyznacza terminy automatycznie na podstawie daty doręczenia i konfigurowalnych reguł, ostrzega o terminach zbliżających się i przekroczonych oraz pozwala odtworzyć chronologię sprawy. System adresowany jest do osób prowadzących własne sprawy oraz do niewielkich podmiotów obsługujących sprawy klientów.

Krótki opis projektu

Sprawnik jest aplikacją webową złożoną z interfejsu przeglądarkowego, serwerowego API oraz relacyjnej bazy danych.

Użytkownik zakłada sprawę i rejestruje w niej pisma przychodzące oraz wychodzące, odnotowując przy każdym dacie nadania i — odrębnie — datę doręczenia. Do pisma dołącza dokumenty. Po wskazaniu reguły właściwej dla danej czynności system wyznacza termin: dodaje do daty doręczenia liczbę dni zapisaną w regule, licząc je w dniach kalendarzowych albo roboczych, a termin przypadający na dzień wolny przesuwają na najbliższy dzień roboczy. Wyznaczony termin zachowuje odniesienie do pisma i reguły, z których wynika, więc pozostaje sprawdzalny.

Terminy prezentowane są w układzie pozwalającym dostrzec te zbliżające się i przekroczone. Reguły wyznaczania terminów są danymi konfiguracyjnymi — dodanie nowej nie wymaga zmiany aplikacji. Projekt ma charakter demonstracyjny i inżynierski: nie odwzorowuje kompletnych procedur administracyjnych, lecz pokazuje działanie mechanizmu na reprezentatywnym zestawie reguł.

Analiza konkurencji

Zadanie, które realizuje Sprawnik, bywa dziś wykonywane czterema sposobami. Żaden z nich nie łączy rejestru korespondencji z regułowym wyznaczaniem terminów od daty doręczenia.

Arkusz kalkulacyjny z kalendarzem. Rozwiązanie najczęstsze wśród osób prowadzących własne sprawy. Zaletą jest zerowy koszt, natychmiastowa dostępność i pełna swoboda układu. Wadą — że termin liczy człowiek. Arkusz nie odróżnia daty nadania od daty doręczenia, nie zna dni ustawowo wolnych i nie przesunie terminu wypadającego w święto. Dokumenty pozostają w folderach, bez powiązania z wierszem arkusza, a przy kilku sprawach równoległych całość przestaje być przeglądalna.

Systemy elektronicznego zarządzania dokumentacją (EZD). Rozbudowane rozwiązania stosowane w urzędach, obsługujące obieg pism, dekretację i archiwum. Zaletą jest kompletność i zgodność z instrukcją kancelaryjną. Wadą — że zbudowano je z perspektywy organu prowadzącego postępowanie, nie strony. Wdrożenie i utrzymanie przekracza możliwości osoby prywatnej lub małego podmiotu, a pilnowanie własnych terminów procesowych nie jest ich zadaniem.

Systemy kancelaryjne dla prawników. Obsługują repertorium spraw, rozliczenia i kalendarz czynności. Zaletą jest dojrzałość i powiązanie sprawy z terminami. Wadą — skierowanie do kancelarii jako organizacji: model licencyjny, zakres funkcji i nakład wdrożeniowy odpowiadają podmiotowi prowadzącemu sprawy zawodowo, a nie osobie prowadzącej kilka spraw własnych.

Ogólne narzędzia do zarządzania zadaniami. Aplikacje z terminami i przypomnieniami. Zaletą jest niski próg wejścia i dobre powiadomienia. Wadą zasadniczą — że termin trzeba do nich wpisać samodzielnie, już wyliczony. Narzędzie nie wie, czym jest data doręczenia, i nie odtworzy, skąd wzięła się data w przypomnieniu.

Kryterium	Arkusz	EZD	Kanc.	Zadania	Sprawnik
Rejestr korespondencji	part.	tak	tak	nie	tak
Odrębna data doręczenia	nie	tak	tak	nie	tak
Wyznaczanie terminu z reguły	nie	nie	part.	nie	tak
Uwzględnianie dni wolnych	nie	nie	part.	nie	tak
Odtwarzalność wyliczenia	nie	nie	nie	nie	tak
Dostępność dla osoby prywatnej	tak	nie	nie	tak	tak

Skróty: *Kanc.* — system kancelaryjny, *Zadania* — ogólny menedżer zadań, *part.* — funkcja obecna częściowo lub zależna od konfiguracji.

Nisza, którą zajmuje projekt, leży między arkuszem a systemem kancelaryjnym: narzędzie na tyle proste, by prowadzić w nim kilka spraw własnych, i na tyle świadome sposobu liczenia terminów, by nie zostawiać tego rachunku człowiekowi.

Wykaz zastosowanych technologii

- Node.js 24.19 — środowisko uruchomieniowe warstwy serwerowej
- TypeScript 5.9 — język warstwy serwerowej i frontendowej
- Express 5.2 — obsługa żądań HTTP i tras REST
- React 19.3 — warstwa frontendowa
- Vite 7.3 — narzędzie budujące i serwer deweloperski frontendu

- PostgreSQL 17.10 — relacyjna baza danych
- pg 8.23 — sterownik bazy danych dla Node.js
- Docker Compose 2.40 — uruchomienie środowiska
- PlantUML 1.2025.4 — diagramy generowane ze źródeł tekstowych

Opis stosu technologicznego i uzasadnienie wybranych technologii

Warstwa frontendowa zbudowana w React komunikuje się z API wyłącznie przez HTTP i format JSON. API zrealizowane w Express przyjmuje żądania, sprawdza token uwierzytelniający i przekazuje operację warstwie dziedzinowej, która korzysta z bazy PostgreSQL przez sterownik pg i pulę połączeń. Pliki dokumentów trafiają na wolumen systemu plików, a w bazie pozostają ich metadane. Całość środowiska opisuje Docker Compose.

Jeden język w obu warstwach. TypeScript po stronie serwera i przeglądarki pozwala opisać te same struktury danych — sprawę, pismo, termin — jednym zestawem typów, zamiast utrzymywać dwa równoległe opisy w różnych językach. Przy projekcie prowadzonym przez jedną osobę ogranicza to koszt przełączania kontekstu, a statyczne typowanie wyłapuje niezgodność formatu odpowiedzi API z oczekiwaniem interfejsu w czasie kompilacji, nie w czasie działania.

PostgreSQL zamiast bazy nierelacyjnej. Dane projektu są z natury relacyjne: sprawa zawiera pisma, pismo rodzi terminy, dokument należy do pisma. Baza relacyjna pozwala wymusić te zależności więzami klucza obcego i ograniczeniami sprawdzającymi, zamiast pilnować ich wyłącznie w kodzie aplikacji. Istotna jest też arytmetyka dat i typy wyliczeniowe, bez których liczenie terminów wymagałoby przenoszenia logiki do warstwy aplikacji. Wybór PostgreSQL wobec innych baz relacyjnych wynika z dojrzałej obsługi typów wyliczeniowych i dat oraz z perspektywy zakresu dodatkowego — wbudowanego wyszukiwania pełnotekstowego, które pozwoli rozszerzyć projekt bez wprowadzania osobnego mechanizmu.

Express zamiast rozbudowanego szkieletu aplikacji. Zakres podstawowy obejmuje kilkanaście tras REST. Szkielet narzucający własną strukturę wniósłby warstwę pojęć przewyższającą złożonością samo zadanie i zaciemniłby podział odpowiedzialności, który projekt ma pokazywać. Express pozostawia ten podział decyzji projektowej, co przy dokumentowanej architekturze jest zaletą.

Docker Compose do uruchomienia bazy. Pozwala odtworzyć środowisko jednym poleceniem i uniezależnia projekt od wersji PostgreSQL zainstalowanej w systemie operacyjnym. Dla pracy ocenianej przez osobę trzecią powtarzalność uruchomienia jest wartością samą w sobie.

2. Kluczowe zagadnienia związane z realizacją projektu

2.1. Przypadki użycia

Aktorzy

Aktor	Opis
Użytkownik	Osoba prowadząca sprawę — własne

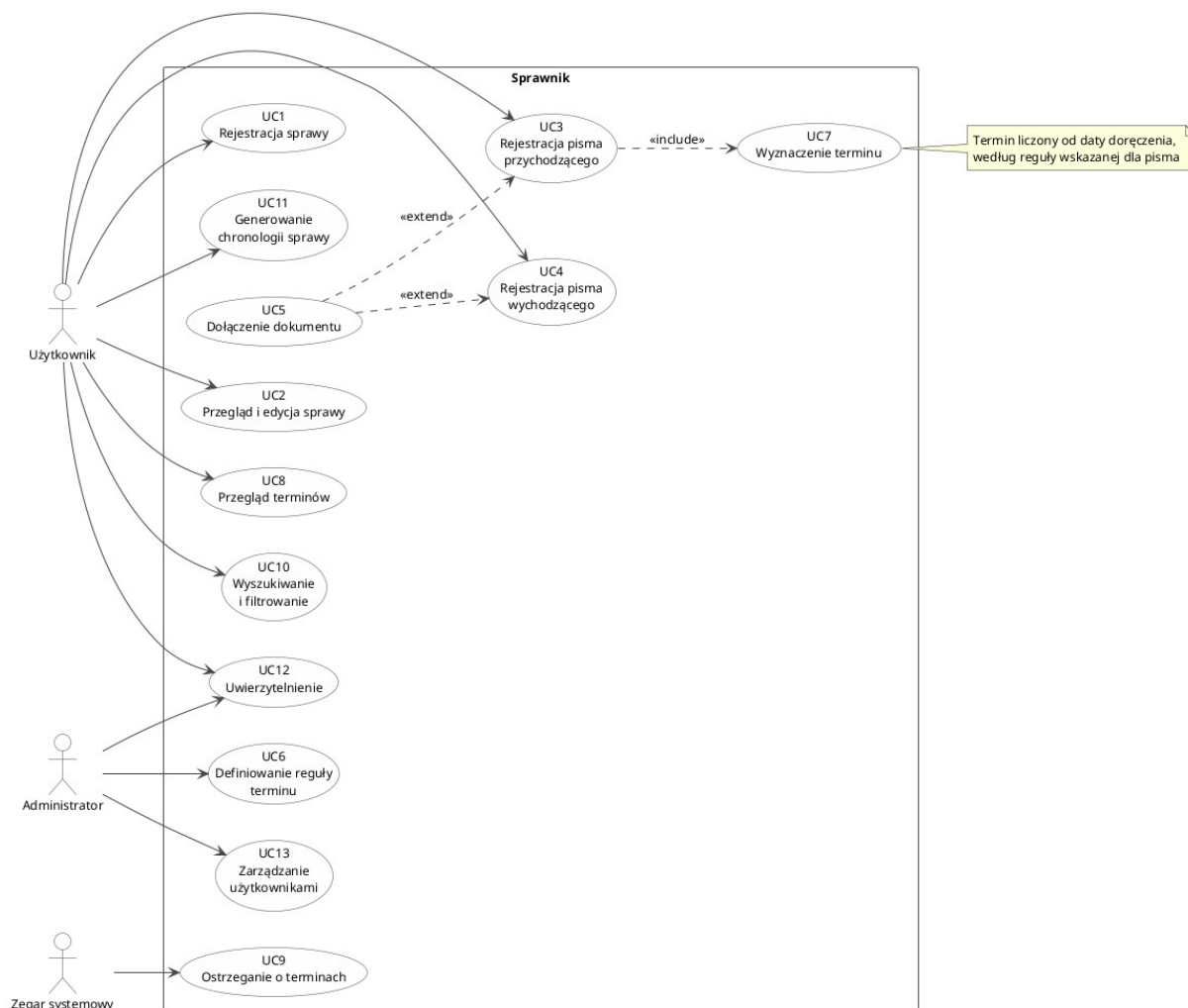
Aktor	Opis
	lub powierzone. Rejestruje sprawy i korespondencję, dołącza dokumenty, korzysta z wyznaczonych terminów i chronologii.
Administrator	Zarządza regułami wyznaczania terminów oraz kontami i uprawnieniami użytkowników. Reguły są danymi konfiguracyjnymi, więc ich definiowanie nie wymaga zmiany aplikacji.
Zegar systemowy	Aktor czasu. Cyklicznie uruchamia ocenę stanu terminów, dzięki czemu ostrzeżenie o terminie zbliżającym się lub przekroczonym powstaje bez udziału użytkownika.

Wykaz przypadków użycia

ID	Przypadek użycia	Aktor	Funkcja
UC1	Rejestracja sprawy	Użytkownik	F1
UC2	Przegląd i edycja sprawy	Użytkownik	F1
UC3	Rejestracja pisma przychodzącego	Użytkownik	F2
UC4	Rejestracja pisma wychodzącego	Użytkownik	F2
UC5	Dołączenie dokumentu	Użytkownik	F3
UC6	Definiowanie reguły terminu	Administrator	F4
UC7	Wyznaczenie terminu	(zawierany przez UC3)	F4
UC8	Przegląd terminów	Użytkownik	F5
UC9	Ostrzeganie o terminach	Zegar systemowy	F5
UC10	Wyszukiwanie i filtrowanie	Użytkownik	F6
UC11	Generowanie chronologii sprawy	Użytkownik	F7
UC12	Uwierzytelnienie	Użytkownik, Administrator	F8

ID	Przypadek użycia	Aktor	Funkcja
UC13	Zarządzanie użytkownikami	Administrator	F8

Kolumna „Funkcja” odsyła do zakresu podstawowego F1–F8 uzgodnionego z promotorem.



Rys. 1. Diagram przypadków użycia — zakres podstawowy.

Dostęp do wszystkich przypadków użycia poza UC12 wymaga uwierzytelnienia. Zależność ta nie została naniesiona na diagram jako trzynasticie relacji «include», ponieważ zaciemniłaby obraz, nie wnosząc informacji.

UC3 — Rejestracja pisma przychodzącego

Przypadek kluczowy dla całego systemu: to w nim powstaje termin, a więc realizuje się główna teza projektu — że punktem odniesienia jest data doręczenia, a nie data nadania.

Aktor: Użytkownik

Warunek wstępny: użytkownik uwierzytelniony, sprawa istnieje w systemie

Przebieg główny

1. Użytkownik wybiera sprawę i wskazuje rejestrację pisma przychodzącego.
2. Podaje nadawcę, znak pisma, rodzaj pisma oraz **datę nadania i datę doręczenia** jako dwie odrębne wartości.
3. Wskazuje regułę terminu właściwą dla czynności, której pismo dotyczy.
4. System zapisuje pismo i przypisuje je do sprawy.
5. System wyznacza termin (UC7) — relacja «include», bo krok wykonuje się zawsze.
6. Wyznaczony termin pojawia się na liście terminów sprawy.

Wynik: pismo zarejestrowane, termin wyznaczony i widoczny w sprawie.

Przebiegi alternatywne

Sytuacja	Zachowanie systemu
Data doręczenia nieznana	Pismo zostaje zapisane bez terminu i oznaczone jako oczekujące na potwierdzenie doręczenia. Termin powstaje dopiero po uzupełnieniu daty — system nie liczy terminu od daty nadania, bo byłoby to liczenie błędne.
Brak reguły dla danej czynności	Użytkownik wprowadza termin ręcznie; pozostaje on oznaczony jako wyznaczony ręcznie, nie automatycznie.
Dokument w postaci pliku	Krok opcjonalny — dołączenie dokumentu (UC5), relacja «extend».

UC7 — Wyznaczenie terminu

Aktor: brak aktora bezpośredniego — przypadek zawierany przez UC3.

Reguła terminu jest zestawem danych, nie fragmentem kodu: określa nazwę, liczbę dni, sposób ich liczenia oraz zdarzenie początkowe. Wyznaczenie terminu polega na zastosowaniu reguły do daty doręczenia:

1. Odczytanie reguły wskazanej dla pisma.
2. Przyjęcie daty doręczenia jako punktu początkowego.
3. Dodanie liczby dni zgodnie ze sposobem liczenia zapisanym w regule (dni kalendarzowe albo robocze).
4. Przesunięcie terminu przypadającego na dzień wolny na najbliższy dzień roboczy — krokami naprzód, aż do dnia, który nie jest ani sobotą, ani niedzielą, ani dniem ustawowo wolnym.
5. Zapisanie terminu wraz z odniesieniem do pisma i reguły, na podstawie których powstał.

Krok 5 jest istotny dla wiarygodności systemu: przy każdym terminie zachowana zostaje informacja, z jakiego pisma i jakiej reguły wynika, dzięki czemu wyliczenie można odtworzyć i sprawdzić.

UC9 — Ostrzeganie o terminach

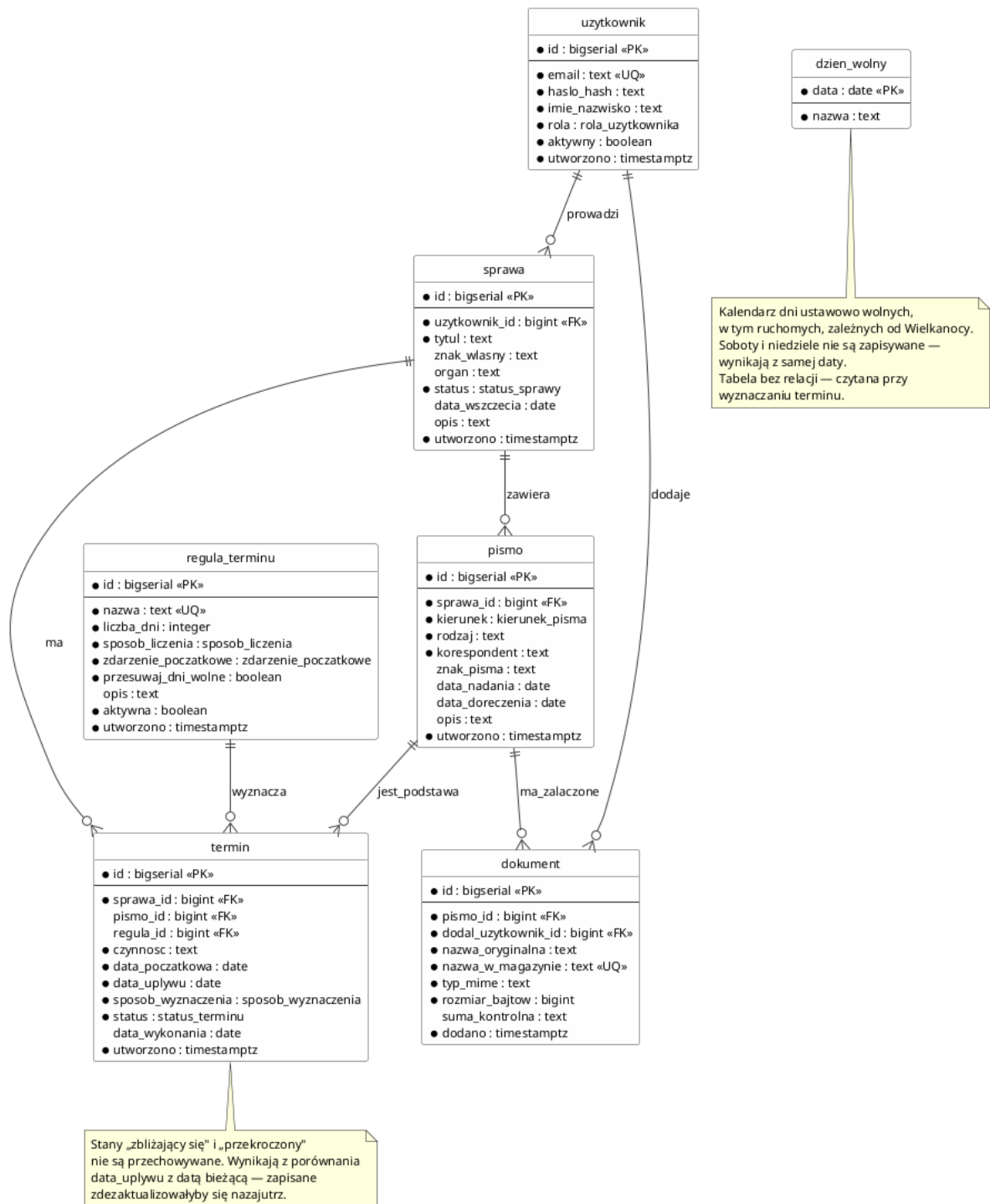
Aktor: Zegar systemowy

Cykliczne przejrzanie terminów otwartych i zakwalifikowanie każdego z nich do jednego ze stanów: odległy, zbliżający się, przekroczony. Progi są konfigurowalne. Ostrzeżenie nie modyfikuje terminu — zmienia wyłącznie sposób jego prezentacji użytkownikowi (UC8).

2.2. Model danych

Model obejmuje siedem tabel. Cztery odwzorowują przedmiot pracy (sprawa, pismo, dokument, termin), jedna użytkowników, a dwie stanowią konfigurację mechanizmu wyznaczania terminów (reguła terminu, kalendarz dni wolnych).

Tabela	Rola w systemie
uzytkownik	Konta i role. Rozróżnienie użytkownika od administratora decyduje o dostępie do reguł terminów i zarządzania kontami.
sprawa	Jednostka nadrzędna. Grupuje korespondencję, dokumenty i terminy jednego postępowania.
pismo	Pojedyncza przesyłka przychodząca lub wychodząca. Nośnik dat, od których liczą się terminy.
dokument	Plik załączony do pisma wraz z metadanymi pozwalającymi zweryfikować jego tożsamość.
regula_terminu	Konfiguracja sposobu wyznaczania terminu. Dane, nie kod.
termin	Wyznaczony termin wraz ze śladem pochodzenia.
dzien_wolny	Kalendarz dni ustawowo wolnych, wykorzystywany przy liczeniu dni roboczych.



Rys. 2. Diagram związków encji (notacja Information Engineering).

Decyzje projektowe

Data nadania i data doręczenia są osobnymi polami, a doręczenie może pozostać nieznane.

To odwzorowanie głównego założenia projektu. Kolumna `pismo.data_doreczenia` dopuszcza wartość pustą — pismo bywa zarejestrowane, zanim potwierdzenie odbioru dotrze do prowadzącego sprawę. Dopóki pozostaje pusta, termin nie powstaje. Alternatywą byłoby liczenie od daty nadania, ale dawałoby to wynik systematycznie занижony, czyli błędny w sposób niebezpieczny dla użytkownika.

Reguła terminu jest wierszem w tabeli, nie gałęzią w kodzie. Tabela `reguła_terminu` opisuje termin czterema cechami: liczbą dni, sposobem ich liczenia, zdarzeniem początkowym oraz tym, czy termin przypadający na dzień wolny przesuwana się na najbliższy roboczy. Dodanie nowej reguły jest operacją na danych i nie wymaga zmiany aplikacji ani jej ponownego wdrożenia.

Termin przechowuje ślad swojego pochodzenia. Oprócz daty upływu zapisywane są: pismo będące podstawą (`pismo_id`), zastosowana reguła (`reguła_id`) oraz data przyjęta za początek biegu (`data_początkowa`). Dzięki temu każde wyliczenie można odtworzyć i sprawdzić, zamiast przyjmować wynik na wiarę. Przy terminie wprowadzonym ręcznie oba odniesienia pozostają puste, a kolumna `sposób_wyznaczenia` odróżnia go od wyznaczonego automatycznie.

Stany „zbliżający się” i „przekroczony” nie są przechowywane. Wynikają z porównania `data_upływu` z datą bieżącą i konfigurowalnym progiem ostrzegania. Zapisane w bazie dezaktualizowałyby się nazajutrz i wymagałyby cyklicznego przeliczania całej tabeli. Przechowywany jest wyłącznie status opisujący decyzję użytkownika: termin otwarty, wykonany albo anulowany.

Kalendarz dni wolnych jest osobną tabelą. Bez niej pojęcia „dni robocze” oraz „przesunięcie terminu z dnia wolnego” nie dają się zdefiniować. `dzień_wolny` nie wchodzi w relacje z pozostałymi tabelami — jest czytana w trakcie wyznaczania terminu.

Tabela przechowuje wyłącznie dni ustawowo wolne. Soboty i niedziele wynikają z samej daty i nie są w niej zapisywane — przechowywanie ich byłoby powielaniem informacji, którą data już niesie. Rozróżnienie to ma znaczenie praktyczne, ponieważ część dni ustawowo wolnych jest ruchoma i zależy od daty Wielkanocy, więc musi zostać wprowadzona do systemu jako dane.

Dokument należy do pisma, a przez nie do sprawy. Rozważono przypisanie dokumentu bezpośrednio do sprawy, co pozwoliłoby przechowywać pliki niezwiązane z żadną przesyłką. Odrzucono je, ponieważ rozmywałoby odpowiedź na pytanie, czego dowodzi dany plik. Obok nazwy pierwotnej zapisywana jest nazwa w magazynie oraz suma kontrolna, co pozwala wykryć podmianę pliku.

Chronologia sprawy nie ma własnej tabeli. Powstaje jako zapytanie łączące pisma, terminy i dokumenty jednej sprawy, uporządkowane datą. Osobna tabela zdarzeń dublowałaby dane już zapisane i wymagałaby utrzymywania ich w zgodzie.

Termin przypadający na dzień wolny

Termin wyznaczony na sobotę, niedzielę lub dzień ustawowo wolny od pracy przesuwana się na najbliższy następny dzień roboczy. Przesunięcie dotyczy wyłącznie dnia końcowego — nie zmienia liczby dni zapisanej w regule ani sposobu ich liczenia.

Istotne jest, że przesunięcie wykonuje się **powtarzalnie, aż do skutku**. Pojedynczy krok naprzód nie wystarcza, ponieważ dzień następujący po dniu wolnym bywa również wolny:

Wyliczona data upływu	Dlaczego jest wolna	Termin ostateczny
sobota	dzień tygodnia	poniedziałek

Wyliczona data upływu	Dlaczego jest wolna	Termin ostateczny
niedziela	dzień tygodnia	poniedziałek
3 maja (środa)	dzień ustawowo wolny	czwartek 4 maja
Wielka Sobota	dzień tygodnia, po niej Niedziela i Poniedziałek Wielkanocny	wtorek po Wielkanocy
25 grudnia	dzień ustawowo wolny, po nim 26 grudnia również	27 grudnia albo pierwszy dzień roboczy po nim
24 grudnia 2026 (czwartek)	Wigilia, Boże Narodzenie, drugi dzień świąt w sobotę i niedziela	poniedziałek 28 grudnia

Dwa ostatnie wiersze pokazują, dlaczego zachowanie trzeba zapisać jako pętlę, a nie jako przesunięcie o jeden dzień: między datą wyliczoną a terminem ostatecznym potrafi leżeć kilka dni wolnych z rzędu.

Wielki Piątek nie występuje w zestawieniu, ponieważ w Polsce nie jest dniem ustawowo wolnym od pracy — ciąg dni wolnych wokół Wielkanocy rozpoczyna się dopiero w sobotę. Kalendarz systemu obejmuje czternaście dni ustawowo wolnych, w tym 24 grudnia, wolne od 2025 roku na podstawie ustawy z 6 grudnia 2024 r. o zmianie ustawy o dniach wolnych od pracy oraz niektórych innych ustaw (Dz.U. 2024 poz. 1965).

Zachowanie to sterowane jest kolumną `regula_terminu.przesuwaj_dni_wolne`. Pozostaje konfigurowalne dla każdej reguły z osobna, ponieważ nie każdy termin umowny musi mu podlegać, natomiast dla reguł odwzorowujących terminy ustawowe przyjmuje wartość prawdziwą.

Typy wyliczeniowe i ograniczenia

Zbiory wartości zamknięte z natury zapisano jako typy wyliczeniowe PostgreSQL: `rola_uzytkownika`, `status_sprawy`, `kierunek_pisma`, `sposob_liczenia`, `zdarzenie_poczkowe`, `sposob_wyznaczenia`, `status_terminu`. Ogranicza to zakres dopuszczalnych wartości na poziomie bazy, a nie wyłącznie aplikacji.

Spójność wspierają ograniczenia sprawdzające. Najistotniejsze z nich:

Ograniczenie	Cel
<code>data_doreczenia >= data_nadania</code>	Doręczenie nie może poprzedzać nadania.
<code>data_uplywu >= data_poczkowa</code>	Termin nie może upływać przed rozpoczęciem biegu.
<code>liczba_dni > 0</code>	Reguła musi wyznaczać termin dodatni.
<code>termin_automatyczny wymaga regula_id i pismo_id</code>	Wyliczenie bez podstawy nie jest

Ograniczenie

data_wykonania tylko przy statusie wykonany

Cel

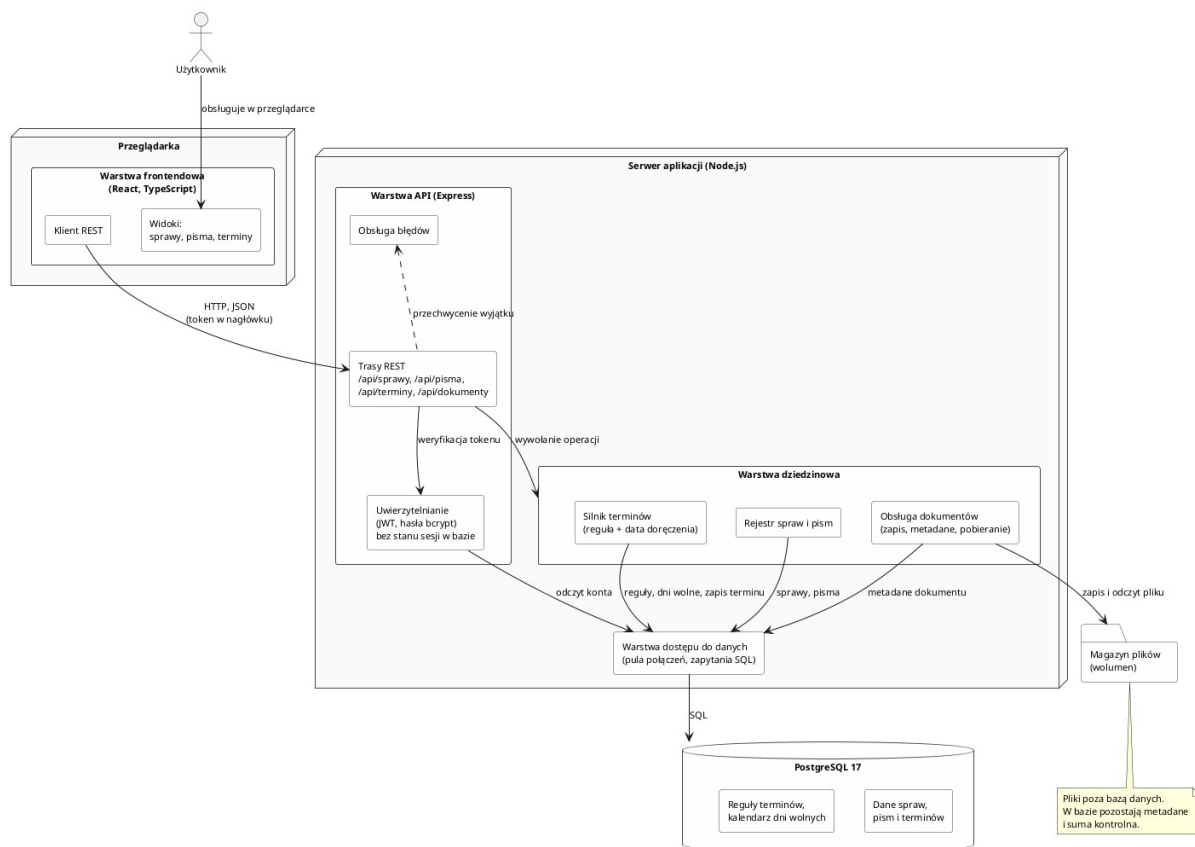
odtworzalne.

Data wykonania bez wykonania jest sprzeczna.

Usunięcie sprawy kasuje kaskadowo jej pisma, dokumenty i terminy, ponieważ poza sprawą tracą one sens. Usunięcie reguły, do której odwołują się terminy, jest blokowane — inaczej znikłaby podstawa wyliczeń już dokonanych.

2.3. Architektura aplikacji

System dzieli się na pięć warstw o rozdzielonych zadaniach. Podział przebiega zarówno przez granicę procesu (przeglądarka, serwer aplikacji, serwer bazy), jak i wewnątrz samej aplikacji serwerowej.



Rys. 3. Diagram komponentów i zależności między nimi.

Warstwa	Zadanie	Nie odpowiada za
Frontendowa	Prezentacja i obsługa interakcji. Wyłącznie komunikacja z API.	Reguły wyznaczania terminów, dostęp do bazy.
API	Przyjęcie żądania, sprawdzenie	Logikę dziedzinową i SQL.

Warstwa	Zadanie	Nie odpowiada za
	uprawnień, walidacja danych wejściowych, zwrócenie odpowiedzi.	
Uwierzytelnianie	Weryfikacja tożsamości i podpisanego tokenu.	Autoryzację operacji dziedzinowych.
Dziedzinowa	Wyznaczanie terminów, rejestr spraw i pism, obsługa dokumentów.	Protokół HTTP i format odpowiedzi.
Dostępu do danych	Pula połączeń i zapytania SQL.	Decyzje o treści danych.

Zależności biegną wyłącznie w jedną stronę: od warstwy zewnętrznej ku bazie. Warstwa dziedzinowa nie zna pojęcia żądania HTTP, a warstwa dostępu do danych nie zna reguł terminów. Dzięki temu silnik terminów daje się sprawdzić testem bez uruchamiania serwera, a wymiana sposobu prezentacji nie narusza reguł wyliczeń.

Rozdzielenie mechanizmu uwierzytelniania

Uwierzytelnianie stanowi odrębny komponent wywoływany przed obsługą żądania, a nie fragment logiki tras. Hasła przechowywane są wyłącznie w postaci skrótu z funkcją bcrypt; system nie dysponuje możliwością odtworzenia hasła.

Sesja jest bezstanowa — tożsamość potwierdza podpisany token przekazywany w nagłówku żądania. W bazie nie powstaje tabela sesji. Rozwiązanie odciąża bazę i upraszcza wdrożenie, ale ma świadomie przyjętą wadę: token pozostaje ważny do czasu wygaśnięcia i nie da się go unieważnić pojedynczo bez dodatkowego rejestru tokenów odwołanych. Dla systemu o zakresie demonstracyjnym jest to kompromis akceptowalny; przy wdrożeniu produkcyjnym wymagałby uzupełnienia.

Rozdzielenie obsługi dokumentów

Pliki przechowywane są poza bazą danych, na wolumenie systemu plików. W bazie pozostają metadane: nazwa pierwotna, nazwa w magazynie, typ, rozmiar i suma kontrolna. Rozważono zapis plików w bazie jako dane binarne — odrzucono go, ponieważ powiększałby kopie zapasowe bazy o treść skanów i obciążał pulę połączeń przesyłaniem dużych obiektów.

Rozdzielenie to wprowadza wymóg spójności między dwoma magazynami: usunięcie dokumentu musi objąć zarówno wiersz, jak i plik. Suma kontrolna pozwala wykryć, że plik w magazynie przestał odpowiadać opisowi w bazie.

Odwzorowanie warstw w kodzie

Podział z diagramu odpowiada strukturze katalogów aplikacji serwerowej, co pozwala odczytać architekturę wprost z repozytorium:

Katalog	Warstwa
backend/src/routes/	API — trasy REST
backend/src/middleware/	uwierzytelnianie, obsługa błędów
backend/src/services/	warstwa dziedzinowa, w tym silnik terminów (w budowie)
backend/src/db/	dostęp do danych
backend/src/config/	konfiguracja ze zmiennych środowiskowych
frontend/src/	warstwa frontendowa

Katalogi oznaczone jako będące w budowie powstają wraz z implementacją kolejnych funkcji. Pozostałe istnieją w repozytorium w chwili przygotowania niniejszego opisu.

Uruchomienie

Baza danych pracuje w kontenerze opisanym w `docker-compose.yml`, co pozwala odtworzyć środowisko jednym poleceniem i uniezależnia projekt od instalacji PostgreSQL w systemie. Parametry połączenia, port oraz sekret podpisujący token pochodzą ze zmiennych środowiskowych — plik `.env.example` zawiera ich wykaz bez wartości rzeczywistych.

2.4. Kluczowe fragmenty implementacji

Przesunięcie terminu na najbliższy dzień roboczy

Zachowanie opisane w punkcie 2.2 zapisane jest jako pętla, a nie jako przesunięcie o jeden dzień. Dzień wolny rozpoznawany jest dwojako: sobota i niedziela wynikają z samej daty, dni ustawowo wolne — z kalendarza w bazie.

```
export function czyDzienWolny(iso: string, kalendarz: Kalendarz):
boolean {
  const dzienTygodnia = naDate(iso).getUTCDay();
  return dzienTygodnia === 0 || dzienTygodnia === 6 ||
kalendarz.has(iso);
}

export function najblizszyDzienRoboczy(iso: string, kalendarz:
Kalendarz): string {
  let data = naDate(iso);
  while (czyDzienWolny(naIso(data), kalendarz)) {
    data = przesun(data, 1);
  }
  return naIso(data);
}
```

Silnik terminów nie odwołuje się do bazy danych ani do protokołu HTTP: przyjmuje regułę, datę początkową i kalendarz jako argumenty. Dzięki temu sprawdzany jest testem

jednostkowym bez uruchamiania serwera — testy obejmują między innymi przypadek czterech dni wolnych z rzędu na przełomie grudnia oraz dni wolne wokół Wielkanocy.

Oprócz daty upływu zwracana jest data sprzed przesunięcia, dzięki czemu wynik można prześledzić zamiast przyjmować go na wiarę.

Powstanie terminu przy rejestracji pisma

Rejestracja pisma i wyznaczenie terminu to jedna operacja niepodzielna, bo UC3 zawiera UC7. Termin powstaje wyłącznie wtedy, gdy nastąpiło zdarzenie, od którego biegnie:

```
const poczatek = dataBiegu(regula.zdarzeniePoczatkowe, pismo);
if (!poczatek) {
  return {
    pismo,
    termin: null,
    powodBrakuTerminu:
      'Pismo oczekuje na potwierdzenie doręczenia – termin powstanie
po uzupełnieniu daty',
  };
}
```

Brak daty doręczenia nie jest więc błędem walidacji, lecz stanem przewidzianym: pismo zapisuje się bez terminu wraz z wyjaśnieniem, a termin powstaje przy uzupełnieniu daty. Alternatywa — przyjęcie daty nadania jako zastępczej — dawałaby wynik systematycznie zaniżony.

Odwzorowanie warstw w kodzie

Każda warstwa z diagramu komponentów ma w repozytorium osobny katalog (routes, middleware, services, db), a zależności biegną wyłącznie w jedną stronę. Trasa nie zawiera zapytań SQL, a moduł dziedzinowy nie zna obiektu żądania. Uwierzytelnianie działa jako ogniwo uruchamiane przed obsługą żądania, więc trasa chroniona nie może pominąć sprawdzenia tożsamości przez przeoczenie.

3. Zrzuty ekranu, wizualizacje

Zrzuty pochodzą z działającej aplikacji wypełnionej danymi demonstracyjnymi. Wszystkie widoczne terminy zostały wyznaczone przez system na podstawie reguł i dat doręczenia — żadnej z tych dat nie wpisano ręcznie.

Sprawnik

Terminy

Sprawy

Roland Piątkowski

Wyloguj

Terminy

Otwarte

Ostrzegaj na 7 dni przed upływem

2 przekroczonych

1 zbliżających się

0 odległych

stan na dzień 21.09.2026

UPŁYWA	CZYNNOŚĆ	SPRAWA	STAN	PODSTAWA
09.09.2026	Złożenie wyjaśnień wraz z dokumentacją	Postępowanie podatkowe — korekta deklaracji	przekroczony o 12 dni	reguła: Odpowiedź na wezwanie do wyjaśnień
14.09.2026	Zgłoszenie uwag do zawiadomienia	Wpis do rejestru zabytków — nieruchomość przy ul. Starej 8	przekroczony o 7 dni	reguła: Uzupełnienie braków formalnych
28.09.2026	Wniesienie odwołania od decyzji odmownej	Odwołanie od decyzji o warunkach zabudowy	zbliża się — 7 dni	reguła: Odwołanie od decyzji

Rys. 4. Przegląd terminów ze wszystkich spraw. Stan terminu wyliczany jest przy odczycie, przez porównanie daty upływu z datą bieżącą, a próg ostrzegania pozostaje nastawialny. Stany odróżnia kolor oraz opis słowny, nie sam kolor.

Sprawnik

Terminy

Sprawy

Roland Piątkowski

Wyloguj

Sprawy

Nowa sprawa

Szukaj po tytule, znaku lub organie

Wszystkie stany

SPRAWA	ORGAN	STAN	PISMA	TERMINY OTWARTE	NAJBLIŻSZY TERMIN
Wpis do rejestru zabytków — nieruchomość przy ul. Starej 8 znak: MWKZ/2026/2	Mazowiecki Wojewódzki Konserwator Zabytków	w toku	1	1	14.09.2026
Postępowanie podatkowe — korekta deklaracji znak: US/2026/3	Naczelnik Urzędu Skarbowego Warszawa-Śródmieście	w toku	2	1	09.09.2026
Odwołanie od decyzji o warunkach zabudowy znak: WZ/2026/14	Urząd Miasta Warszawy, Wydział Architektury	w toku	2	1	28.09.2026

Rys. 5. Rejestr spraw z wyszukiwaniem i filtrowaniem. Liczba pism, liczba terminów otwartych i data najbliższego terminu powstają w jednym zapytaniu do bazy.

Sprawnik
Terminy
Sprawy

Roland Piątkowski
Wyloguj

← Sprawy

Zarejestruj pismo

Odwołanie od decyzji o warunkach zabudowy

Urząd Miasta Warszawy, Wydział Architektury · znak WZ/2026/14 · w toku · wszczęcie 03.08.2026

Terminy

CZYNNOŚĆ	BIEG OD	UPŁYWA	STAN	PODSTAWA	
Wniesienie odwołania od decyzji odmownej	12.09.2026	28.09.2026	zbliża się — 7 dni	reguła: Odwołanie od decyzji	Wykonany Anuluj

Korespondencja

KIERUNEK	RODZAJ	KORRESPONDENT	ZNAK	NADANIE	DORĘCZENIE
wychodzące	Wniosek o wydanie kopii akt	Urząd Miasta Warszawy	—	14.09.2026	oczekuje — potwierdź
przychodzące	Decyzja o odmowie ustalenia warunków zabudowy	Urząd Miasta Warszawy	AM-WZ.6730.214.2026	09.09.2026	12.09.2026

Chronologia sprawy

- 12.09.2026 Wpłynęło: Decyzja o odmowie ustalenia warunków zabudowy — Urząd Miasta Warszawy
- 14.09.2026 Wysłano: Wniosek o wydanie kopii akt — Urząd Miasta Warszawy
- 28.09.2026 Termin: Wniesienie odwołania od decyzji odmownej

Rys. 6. Widok sprawy: terminy wraz ze wskazaniem reguły, z której powstały, korespondencja oraz chronologia. Pismo bez potwierdzonego doręczenia ma w kolumnie „doręczenie” przycisk uzupełnienia daty — dopiero ta czynność wyznacza termin.

Sprawnik

Terminy

Sprawy

Roland Piątkowski

Wyloguj

← Sprawy

Anuluj

Odwołanie od decyzji o warunkach zabudowy

Urząd Miasta Warszawy, Wydział Architektury · znak WZ/2026/14 · w toku · wszczęcie 03.08.2026

Rejestracja pisma

Kierunek

Przychodzące

Rodzaj pisma

Postanowienie o zawieszeniu pc

Nadawca

Urząd Miasta Warszawy

Znak pisma

AM-WZ.6730.214.2026/3

Data nadania

15.09.2026

Data doręczenia

18.09.2026

Reguła terminu

Zażalenie na postanowienie — 7 dni

Czynność (domyślnie nazwa reguły)

Termin liczony od daty doręczenia; termin przypadający na dzień wolny przesuwa się na najbliższy dzień roboczy. Bez daty doręczenia pismo zapisze się bez terminu.

Zapisz pismo

Terminy

CZYNNOŚĆ	BIEG OD	UPLYWA	STAN	PODSTAWA	
Wniesienie odwołania od decyzji odmownej	12.09.2026	28.09.2026	zbliża się — 7 dni	reguła: Odwołanie od decyzji	Wykonany Anuluj

Rys. 7. Rejestracja pisma przychodzącego (UC3). Data nadania i data doręczenia są odrębnymi polami, a formularz uprzedza, że bez daty doręczenia pismo zapisze się bez terminu.

4. Wnioski i perspektywy rozwoju

Etap pierwszy zamyka się działającą aplikacją, a nie samym projektem na papierze. Zrealizowane zostały: rejestr spraw i korespondencji, automatyczne wyznaczanie terminów według reguł, ostrzeganie o terminach zbliżających się i przekroczonych, chronologia sprawy oraz uwierzytelnianie. Przejście całej ścieżki — od zalogowania, przez rejestrację pisma, po odczytanie terminu, który z niego powstał — jest możliwe w interfejsie.

Trzy wnioski wynikają wprost z przebiegu prac.

Rozdzielenie daty nadania i daty doręczenia okazało się decyzją, wokół której ułożył się cały model. Nie jest to szczegół zapisu: pociągnęło za sobą dopuszczenie pustej daty doręczenia, stan pisma oczekującego na potwierdzenie odbioru oraz regułę, że termin powstaje dopiero po zdarzeniu początkowym. Gdyby system liczył od daty nadania, byłby szybszy w budowie i systematycznie zaniżałby terminy — czyli mylił się w kierunku niebezpiecznym dla użytkownika.

Zapisanie reguły terminu jako danych, a nie jako gałęzi w kodzie, sprawdziło się w praktyce. Siedem reguł startowych, w tym liczona w dniach roboczych i nieprzesuwana z dnia wolnego, powstało bez jednej zmiany w aplikacji. Dodanie kolejnej pozostaje operacją na danych.

Wydzielenie silnika terminów jako funkcji niezależnej od bazy i od protokołu HTTP pozwoliło sprawdzić testem sytuacje trudne do odtworzenia ręcznie — kilka dni wolnych z rzędu, dni ruchome zależne od daty Wielkanocy, przełom grudnia. Test ujawnił przy okazji błąd w założeniu przyjętym wcześniej: Wielki Piątek, wskazany pierwotnie jako dzień ustawowo wolny, takim dniem w Polsce nie jest. Opis i dane rozeszły się w miejscu, które bez testu pozostałoby niezauważone aż do pierwszego terminu wypadającego w kwietniu.

Ograniczenia obecnego stanu

Dołączanie dokumentów ma gotowy model danych, ale obsługa plików nie została jeszcze zaimplementowana. Sesja bezstanowa nie pozwala unieważnić pojedynczego tokenu przed jego wygaśnięciem. Kalendarz dni ustawowo wolnych wprowadzono do roku 2030 i wymaga uzupełniania. Zestaw reguł jest reprezentatywny, a nie kompletnym odwzorowaniem procedur administracyjnych.

Kierunki dalszych prac

Najbliższe: obsługa plików wraz z sumą kontrolną i powiązaniem dokumentu z pismem, panel zarządzania kontami dla administratora oraz cykliczna ocena stanu terminów wraz z powiadomieniem, dziś wykonywana przy odczycie. Dalsze, ujęte w zakresie dodatkowym: wyszukiwanie pełnotekstowe w treści dokumentów, panel statystyk i eksport kompletnych akt sprawy.

5. Bibliografia/źródła

Akty prawne

1. Ustawa z dnia 14 czerwca 1960 r. — Kodeks postępowania administracyjnego (Dz.U. 1960 nr 30 poz. 168, z późn. zm.; tekst jedn. Dz.U. 2025 poz. 1691).
2. Ustawa z dnia 30 sierpnia 2002 r. — Prawo o postępowaniu przed sądami administracyjnymi (Dz.U. 2002 nr 153 poz. 1270, z późn. zm.; tekst jedn. Dz.U. 2024 poz. 935).
3. Ustawa z dnia 18 stycznia 1951 r. o dniach wolnych od pracy (Dz.U. 1951 nr 4 poz. 28, z późn. zm.; tekst jedn. Dz.U. 2025 poz. 296).
4. Ustawa z dnia 6 grudnia 2024 r. o zmianie ustawy o dniach wolnych od pracy oraz niektórych innych ustaw (Dz.U. 2024 poz. 1965) — wprowadza 24 grudnia jako dzień ustawowo wolny od pracy, obowiązuje od 1 lutego 2025 r.

Sygnatury zweryfikowano w Internetowym Systemie Aktów Prawnych (isap.sejm.gov.pl). Akty przywołano jako źródło reguł odwzorowanych w systemie w postaci danych. Projekt ma charakter demonstracyjny i nie stanowi odwzorowania kompletnych procedur administracyjnych.

Dokumentacja techniczna

5. *PostgreSQL 17 Documentation*, The PostgreSQL Global Development Group, <https://www.postgresql.org/docs/17/>
6. *React Documentation*, Meta Open Source, <https://react.dev>
7. *Express 5.x API Reference*, OpenJS Foundation, <https://expressjs.com>

8. *Vite Guide*, <https://vite.dev>
9. *node-postgres* — *PostgreSQL client for Node.js*, <https://node-postgres.com>
10. *Zod* — *TypeScript-first schema validation*, <https://zod.dev>

Pozostałe źródła

11. N. Provos, D. Mazières, *A Future-Adaptable Password Scheme*, Proceedings of the USENIX Annual Technical Conference, 1999 — opis funkcji bcrypt zastosowanej do przechowywania haseł.
12. M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force, 2015.
13. *OWASP Cheat Sheet Series — Password Storage*, OWASP Foundation, <https://cheatsheetseries.owasp.org>
14. J. Meeus, *Astronomical Algorithms*, wyd. 2, Willmann-Bell, 1998 — algorytm wyznaczania daty Wielkanocy wykorzystany przy budowie kalendarza dni ruchomych.
15. *Szablon dokumentacji projektowej*, Uniwersytet VIZJA — wzór układu i wymagań formalnych niniejszego dokumentu.